

How `sitl_init.sh` works

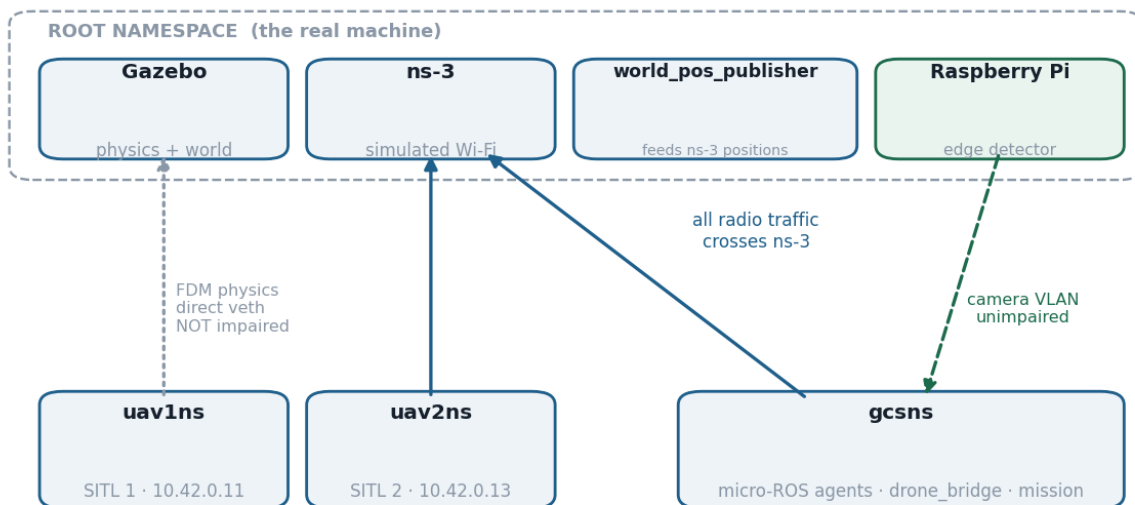
One script builds a two-aircraft hardware-in-the-loop testbed: two simulated drones, a ground station, a simulated radio channel, and a real Raspberry Pi doing the vision work. This is what it assembles, in what order, and why the order matters.

The problem it solves

A multi-UAV experiment needs four things running at once, and they all want to talk to each other: **Gazebo** for physics, **ArduPilot SITL** for the autopilot, **ns-3** for the radio, and **ROS 2** to carry commands and telemetry. Left to themselves on one machine they would simply talk over loopback at full speed — and the network simulation would be bypassed without anyone noticing.

The launcher's job is to make that impossible. It puts each aircraft in its own **Linux network namespace**, so the only route between them runs through ns-3's simulated Wi-Fi. If the plumbing is wrong, nothing connects at all — which is much safer than silently measuring a perfect network.

What the launcher builds



A namespace is a private network stack: its own interfaces, routes, ports and loopback. These three can only reach each other through ns-3.

That split is the whole point. The CAMERA link is a cable and must stay clean; the RADIO link is the thing under study and must cross the simulated channel.

Three isolated worlds. Everything above the dashed line is the real machine; the three boxes below are private network stacks that can only reach one another through the simulated channel.

Why a network namespace

A namespace gives a group of processes their own interfaces, routes, firewall rules, port numbers and loopback. It is the same kernel feature Docker is built on, used here directly. Two consequences matter:

- 1. Ports stop colliding.** Every ArduPilot SITL wants the same default ports. In one shared namespace you must hand-allocate them and hope; in separate namespaces each aircraft reuses identical numbers, exactly as separate physical drones would.
- 2. Traffic cannot take a shortcut.** A fresh namespace has no route to anything. The only path out is the one the script builds, and that path goes through ns-3.

OK A namespace's loopback is private too, which is why `-sim-address` must be a real IP and never `127.0.0.1`: the two sides live in different namespaces and do not share a loopback.

The network fabric

Each namespace is wired to ns-3 through the same four-part chain. A **veth** is a virtual cable with two ends; one end is pushed inside the namespace, the other stays outside. A **bridge** is a virtual switch. A **TAP** is a device ns-3 can attach to and read raw frames from.

How one telemetry packet reaches the ground station



Only the shaded hop applies delay, loss and fading. Everything else is plain Linux plumbing whose job is to force the packet through it.

Remove any one link and the two namespaces simply cannot talk — which is what makes the measurement honest.

Every hop a telemetry packet takes from the autopilot to the ground station. Nine of the ten are ordinary Linux plumbing; only one applies the physics being studied.

Node numbering

ns-3 knows nothing about aircraft — it has numbered nodes. The mapping interleaves aircraft with their companion computers, because on a real airframe the Pi flies bolted to the autopilot:

ns-3 node ids: aircraft and companion computers interleave

	role	TAP	address	lives in
node 0	GCS	tap-gcs	10.42.0.10	gcsns
node 1	SITL 1	tap-uav1	10.42.0.11	uav1ns
node 2	Pi 1	tap-uav2	10.42.0.12	hardware
node 3	SITL 2	tap-uav3	10.42.0.13	uav2ns
node 4	Pi 2	tap-uav4	10.42.0.14	hardware

A companion computer sits on the SAME airframe as the aircraft above it, so node 2 mirrors node 1's position and node 4 mirrors node 3's.

Node ids are shared by ns-3, the position publisher and the obstacle model. Getting one wrong makes a link's measurements fiction while everything still appears to run.

The two links to the Raspberry Pi

The Pi has one Ethernet port but needs two logically separate links, so the script uses 802.1Q VLAN tagging to split the single cable:

	VLAN 10 — camera	VLAN 42/43 — radio
Address	10.0.0.x	10.42.0.x
Carries	camera frames <i>into</i> the Pi	detections <i>out</i> of the Pi
Path	direct to Gazebo, never enters a bridge ns-3 can see	br-uav2 → tap-uav2 → ns-3 → tap-gcs → gcsns
Impaired?	No — it stands in for the ribbon cable inside one airframe	Yes — this is the radio under study

The routing then enforces itself: Gazebo can only reach the Pi at 10.0.0.2 and the ground station only at 10.42.0.12, so neither can take the wrong path even by accident.

Startup, stage by stage

Eleven stages run in a fixed order. Three of them are hard ordering constraints rather than preferences — get those wrong and the run fails in ways that look like something else entirely.

Startup order — and why it cannot be shuffled



Shaded steps are ordering constraints, not preferences: ns-3 must hold the TAPs before traffic flows, the agents must be listening before SITL dials in, and the gates must pass before a mission is allowed to arm anything.

The launch ladder. Shaded steps must happen when they happen; the others are merely sensible.

The three constraints

ns-3 before any traffic (stage 3). ns-3 must own the TAP devices before anything tries to send. The script waits for carrier on all four TAPs and aborts if ns-3 exits first.

Agents before SITL (stage 5). ArduPilot's DDS client dials out to a fixed address. If nothing is listening, it retries and may give up; the agents must already be bound.

Gates before flying (stage 7). The script waits for a real navsat message on *both* aircraft — not merely a publisher count, which can read healthy while nothing arrives.

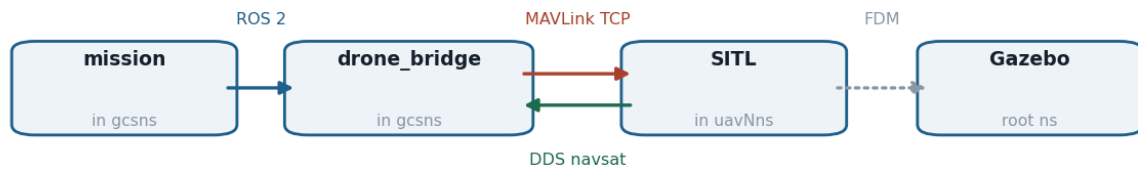
Every address and port in one place

	UAV1	UAV2	carried over
Namespace	uav1ns	uav2ns	—
Radio address	10.42.0.11	10.42.0.13	ns-3 Wi-Fi
MAVLink (commands)	TCP 5760	TCP 5770	ns-3 Wi-Fi
DDS to agent (telemetry)	UDP 2019	UDP 2020	ns-3 Wi-Fi
Gazebo FDM (physics)	UDP 9002	UDP 9012	direct veth
Management link	172.31.1.1 / .2	172.31.2.1 / .2	direct veth
Raspberry Pi radio	10.42.0.12	10.42.0.14	ns-3 Wi-Fi
Raspberry Pi camera	10.0.0.2	10.0.1.2	VLAN, unimpaired

The ground station sits at 10.42.0.10 inside gcsns and runs both micro-ROS agents, both drone_bridge nodes and the mission scripts. Note the pattern in the last three rows: anything to do with *flying* avoids ns-3, and anything to do with *communicating* goes through it.

What happens once it is running

Three separate conversations happen at once



■ commands out (arm, takeoff, goto) travel MAVLink over TCP - through ns-3

■ telemetry back (position, battery) travels DDS - also through ns-3

■ flight physics uses a direct veth and NEVER touches ns-3

Physics is deliberately exempt: impairing it would change how the aircraft FLIES, when the experiment is about how it COMMUNICATES.

Three conversations at once. Commands and telemetry cross the simulated channel; flight physics deliberately does not.

Reading the output

Line you should see	What it confirms
All TAPs attached	ns-3 owns the devices; the simulated channel exists
/ap/v1/navsat is delivering messages	real telemetry, not just a registered publisher
Letting AP_DDS settle (15s)	handshake headroom before anything talks to SITL
PIPELINE READY	safe to fly a mission

Two things that bite

! Duplicate node names. Both bridges are the same executable, so without `-r __node:=drone_bridge_uavN` both register as `/drone_bridge`. Two ROS 2 nodes sharing a name is undefined behaviour: the one that starts second displaces the first, so UAV1 silently loses its topics — every run, never UAV2.

! A comment after a line continuation. Bash joins the next line literally, so a `#` there comments out the arguments. It cost a run where the position publisher came up on its defaults and nobody could see why.

Running it

```
# the full two-board run, with cameras and the Gazebo viewer
./scripts/netns/run_hitl.sh --uavs 2 --view --gui --mission

# host only, no Raspberry Pi attached
./scripts/netns/run_hitl.sh --uavs 2 --no-pi --mission

# tear everything down
sudo bash scripts/netns/kill_all_netns.sh
```

Logs live in `/tmp`: `ns3_2uav.log`, `gazebo_2uav.log`, `agent_2uav_uavN.log`, `bridge_2uav_uavN.log`, `hitl_mission_uavN.log`. When a mission misbehaves, read them in that order — it follows the direction the data flows.